Python code documentation for the article *Diagnosing bias in small biomedical image datasets through cross-task post-hoc explanations*, C. Bolut et al.

June 7, 2026

Contents

1 Color coding

- Pink for files corresponding to tissue repair binary classification (regeneration or scarring).
- Blue for files corresponding to mice classification.
- Brown for files shared by the two tasks.

2 Neural networks

2.1 Images distribution into folds

Creation of a `txt` file that contains a Python matrix of five lines and several columns. Line i contains the names of the images placed into fold i. All the images of the given folder are distributed.

Python file: `build_folds.py`	
Input	Output
Folder with `tiff` images.	Text file with the images names distributed into the five folds: `folds5[...].txt`.

2.2 `h5` file creation

The previously generated text file enables to create an `h5` file that contains the training and validation sets images organized along the five given folds. Another `h5` file is created for the test set images. Files contain both the images and their corresponding labels. The label associated to each image depends on the considered task:

- for binary tissue repair classification, the `False` label corresponds to "regenerating" and `True` to "scarring".
- for mice classification, label s corresponds to mouse number s.

Python files: `train_ch12_allNets_fldbyfld.py` / `mice_classif_resnet_fldbyfld.py` or `mice_classif_otherNet_fldbyfld.py`	
Input	Output
• Folder with the `tiff` images. • Text file with the images distributed into the five folds: `folds5[...].txt`.	• Text file summarizing the creation of the `h5` file: `train_ch12[...]validfld`i`[...]createh5.txt` / `mice[...]validfld`i`[...]createh5.txt`. • `h5` file containing the images according to the prescribed fold distribution: `dataset[...]5folds[...].h5` / `mice[...]5folds[...]`n`mice.h5`.
For mice classification, the `otherNet` file is the implementation for the SqueezeNet and vgg neural networks.	
The files contain the implementation of `h5` files creation and of training; the appropriate section must be uncommented.	

2.3 Learning

2.3.1 Training

Training of the three neural networks based on the previously created `h5` file, while designating each time a different fold for validation. These five trainings can be computed in parallel, validation fold by validation fold ("fldbyfld"), which produces five models trained on slightly different datasets.

During each training, the model's weights are saved at its best performance (minimum validation loss function) and at the end of the training, so that a training can be interrupted and then taking up again.

Python files: `train_ch12_allNets_fldbyfld.py` / `mice_classif_resnet_fldbyfld.py` or `mice_classif_otherNet_fldbyfld.py`

Input	Output
• `h5` file containing the training and validation images according to the prescribed fold distribution: `dataset[...]5folds[...].h5` / `mice[...]5folds[...]`*n*`mice.h5`. • Text file with the images distributed into the five folds: `folds5[...].txt`.	• Text file summarizing the training: `train_ch12[...]validfld`*i*`[...].txt` / `mice[...]validfld`*i*`[...].txt`. • `tar` files with model backup (best performance and end of training): `best_models/fold`*i*`_ep`*n*`.tar` and `checkpoints/fold`*i*`_ep`*m*`.tar`. • `json` file with the values of the loss function and accuracy at every epoch: `train_ch12[...]validfld`*i*`[...].json` / `mice[...]validfld`*i*`[...].json`. • `png` files with the plots of the loss function and accuracy: `train_ch12[...]validfld`*i*`[...]loss.png`, `train_ch12[...]validfld`*i*`[...]acc.png` / `mice[...]validfld`*i*`[...]loss.png`, `mice[...]validfld`*i*`[...]acc.png`.

2.3.2 Evaluation

Using confusion matrices to evaluate the five neural networks obtained at the previous step on the test set, with detailed metrics indicated.

Python files: plot_binary_confusion_matrix_allNets.py / plot_mice_confusion_matrix_resnet.py or plot_mice_confusion_matrix_otherNet.py	
Input	Output
• h5 file of the test set: dataset[...]Te[...]5folds[...].h5 / mice[...]Te[...]5folds[...]*n*mice.h5. • tar backup file of the model at its best performance: best_models/fold*i*_ep*n*.tar.	• Confusion matrix on the test set: train_ch12[...]validfld*i*[...]confmatReport_Te.png / mice[...]testfld*i*[...]confusionmatrix.png. • Print of the complete evaluation report (with precision, recall, f1-score and support).

3 XGBoost

Data available: n labelled images of the tiff format (each image corresponds to a specific mouse): $n = n_{train} + n_{test}$ with the training and test sets.

3.1 Features extraction

Using the PyFeats library to go from n images to features stored in n pkl files, with m features per file.

Python file: features_extraction_chdb.py	
Input	Output
Folder with tiff images.	• For each parallel worker, a text file summarizing the extraction: features_chdb[...].txt. • Storage folder of the extracted features contained in the pkl files: output_chdb[...].

Note: this code can also be used to generate the file containing the names of all features (pyfeats_features_names_230623.pkl).

3.2 Features selection

Going from m to $m' < m$ features.

3.2.1 Selection on the training set

Using the Boruta algorithm to go from n_{train} pkl files to two npy files, the first one containing a $n_{train} \times m'$ matrix and the second one the mask of the m' selected features.

The labels of the images (in the ytrain and ytest[...].npy files) depend on the task considered:

- for binary tissue repair classification, label 0 corresponds to "regenerating" and 1 to "scarring".
- for mice classification, label s corresponds to mouse number s.

Python files: `binclassif_varselection.py` / `miceclassif_varselection.py`	
Input	Output
• Folder with the extracted features of the training set in the `pkl` format. • File with the names of the m features: `pyfeats_features_names_230623.pkl`.	• Text file summarizing the selection: `binary_classif_ML[...].txt` / `miceclassif_ML[...].txt`. • File with the mask of the selected features (of size m, indicating the m' selected features): `featmask_binary_classif_ML[...].npy` / `featmask_miceclassif_ML[...].npy`. • File with the matrix $n_{train} \times m'$: `Xtrainfiltered_binary_classif_ML[...].npy`/ `Xtrainfiltered_miceclassif_ML[...].npy` (or `Xfiltered_miceclassif_ML[...].npy`). • File with the labels of the n_{train} images of the training set: `ytrain_binary_classif_ML[...].npy` / `ytrain_miceclassif_ML[...].npy`.

3.2.2 Application to the test set

Creation for the test set of a `npy` file with a $n_{test} \times m'$ matrix, from the m' features selected beforehand.

Python files: `binclassif_varselection.py` / `miceclassif_varselection.py`	
Input	Output
• Folder with the extracted features of the test set in the `pkl` format. • File with the mask of the selected features: `featmask_binary_classif_ML[...].npy` / `featmask_miceclassif_ML[...].npy`.	• Text file summarizing the selection: `binary_classif_ML[...].txt` / `miceclassif_ML[...].txt`. • File with the matrix $n_{test} \times m'$: `Xtestfiltered_binary_classif_ML[...].npy`/ `Xtestfiltered_miceclassif_ML[...].npy`. • File with the labels of the n_{test} images of the test set: `ytest_binary_classif_ML[...].npy` / `ytest_miceclassif_ML[...].npy`.

3.3 Learning

3.3.1 Training

A trained XGBoost model is obtained from the m' features for each of the n_{train} images.

Python files: binclassif_MLShap.py / miceclassif_MLShap.py	
Input	Output
• File with the matrix $n_{train} \times m'$: Xtrainfiltered_binary_classif_ML[...].npy/ Xtrainfiltered_miceclassif_ML[...].npy. • File with the labels of the n_{train} images of the training set: ytrain_binary_classif_ML[...].npy / ytrain_miceclassif_ML[...].npy.	• Text file summarizing the training: binary_classif_ML[...]xgboost.txt / miceclassif_ML[...]xgboost.txt. • File storing the trained XGBoost model: trainedXGBoost[...] binary_classif_ML[...].pkl / trainedXGBoost[...]miceclassif_ML[...].pkl.
A preliminary step of looking for XGBoost's optimal parameters with GridSearch can be added.	

3.3.2 Evaluation

Using confusion matrices to evaluate, on the training and test sets, the XGBoost model obtained at the previous step.

Python files: binclassif_MLShap.py / miceclassif_MLShap.py	
Input	Output
• File storing the trained XGBoost model: trainedXGBoost[...] binary_classif_ML[...].pkl / trainedXGBoost[...] miceclassif_ML[...].pkl. • File with the matrix $n_{train} \times m'$: Xtrainfiltered_binary_classif_ML[...].npy/ Xtrainfiltered_miceclassif_ML[...].npy. • File with the labels of the n_{train} images of the training set: ytrain_binary_classif_ML[...].npy / ytrain_miceclassif_ML[...].npy. • File with the matrix $n_{test} \times m'$: Xtestfiltered_binary_classif_ML[...].npy/ Xtestfiltered_miceclassif_ML[...].npy. • File with the labels of the n_{test} images of the test set: ytest_binary_classif_ML[...].npy / ytest_miceclassif_ML[...].npy.	• Text file summarizing the training: binary_classif_ML[...]matconf.txt / miceclassif_ML[...]matconf.txt. • Confusion matrixon the training or test set: binary_classif_ML[...]confmat[...].png / miceclassif_ML[...]confmat[...].png. • Print of the complete evaluation report (with precision, recall, f1-score and support).

3.4 Explainability

Using SHAP to obtain features ranked by importance order for mice classification.

3.4.1 Computation of SHAP values

Python files: binclassif_MLShap.py / miceclassif_MLShap.py	
Input	Output
• File storing the trained XGBoost model: trainedXGBoost[...] binary_classif_ML[...].pkl / trainedXGBoost[...] miceclassif_ML[...].pkl. • File with the matrix $n_{train} \times m'$: Xtrainfiltered_binary_classif_ML[...].npy/ Xtrainfiltered_miceclassif_ML[...].npy. • File with the matrix $n_{test} \times m'$: Xtestfiltered_binary_classif_ML[...].npy/ Xtestfiltered_miceclassif_ML[...].npy.	• Text file summarizing the training: binary_classif_ML[...]saveshap.txt / miceclassif_ML[...]saveshap.txt. • File storing the SHAP values: SHAPvalues_binary_classif_ML[...].npy / SHAPvalues_miceclassif_ML[...].npy.

3.4.2 SHAP plots

Python files: binclassif_MLShap.py / miceclassif_MLShap.py	
Input	Output
• File storing the SHAP values: SHAPvalues_binary_classif_ML[...].npy / SHAPvalues_miceclassif_ML[...].npy. • File with the matrix $n_{train} \times m'$: Xtrainfiltered_binary_classif_ML[...].npy/ Xtrainfiltered_miceclassif_ML[...].npy. • File with the matrix $n_{test} \times m'$: Xtestfiltered_binary_classif_ML[...].npy/ Xtestfiltered_miceclassif_ML[...].npy. • File with the names of the m features: pyfeats_features_names_230623.pkl. • File with the mask of the selected features: featmask_binary_classif_ML[...].npy / featmask_miceclassif_ML[...].npy.	• Text file summarizing the training: binary_classif_ML[...]plotshap.txt / miceclassif_ML[...]plotshap.txt. • SHAP plots: binary_classif_ML[...]SHAPlocal[...].png/ miceclassif_ML[...]SHAPlocal[...]mices.png (for mouse s).

3.4.3 Summary of the SHAP analyses (for mice classification only)

Part one: from SHAP values to ranked features From the SHAP values, generation of the matrix of ranked features. This matrix has as many lines as there are mice; on each line is the ranked list (according to SHAP) of all the features, identified by their number. Element at row s and column j is the number of the jth most important feature according to SHAP for the prediction of mouse s.

Python file: miceclassif_Shap_analysis.py	
Input	Output
File storing the SHAP values: SHAPvalues_miceclassif_ML[...].npy.	File storing the matrix of ranked features: SHAPmatfeat_miceclassif_ML[...].npy.

Part two: plots Using the matrix of ranked features to plot several figures summarizing the information from SHAP about mice classification.

Python file: miceclassif_Shap_analysis.py	
Input	Output
• File with the names of the m features: pyfeats_features_names_230623.pkl • File with the mask of the selected features: featmask_miceclassif_ML[...].npy. • File storing the matrix of ranked features: SHAPmatfeat_miceclassif_ML[...].npy.	• Figure with two graphs: average position in the SHAP plots and number of occurences in the SHAP top 20 (with colored classes), for a prescribed number v of features: SHAP[...]meanocc_v_4classes.png. • Graph where each feature is placed within the graph with x-axis "average position in the SHAP plots" and y-axis "number of occurences in the SHAP top 20": SHAP[...]meanocc_x0tox$_{\text{max}}$.png.

3.4.4 Dependence plots (DP)

Ploting the graphs

Python files: binclassif_ShapQuanti.py / miceclassif_ShapQuanti.py	
Input	Output
• File storing the SHAP values: SHAPvalues_binary_classif_ML[...].npy / SHAPvalues_miceclassif_ML[...].npy. • File with the matrix $n_{train} \times m'$: Xtrainfiltered_binary_classif_ML[...].npy/ Xtrainfiltered_miceclassif_ML[...].npy. • File with the labels of the n_{train} images of the training set: ytrain_binary_classif_ML[...].npy / ytrain_miceclassif_ML[...].npy • File with the matrix $n_{test} \times m'$: Xtestfiltered_binary_classif_ML[...].npy/ Xtestfiltered_miceclassif_ML[...].npy. • File with the labels of the n_{test} images of the test set: ytest_binary_classif_ML[...].npy / ytest_miceclassif_ML[...].npy • File with the names of the m features: pyfeats_features_names_230623.pkl. • File with the mask of the selected features: featmask_binary_classif_ML[...].npy / featmask_miceclassif_ML[...].npy.	• For tissue repair binary classification: DP of a given feature j and its influence on the prediction of the "regenerating" class (function dp_binary): Shap_DepPlot_feat$j$/Shap_DepPlot_ binclassif_ML[...]feat$j$[...].png. ↪ Coloring of the training and test sets (function dp_binary_traintest): similar with the suffix TnTe. ↪ Coloring of the predicted class ("regenerating") in relation to the other (function dp_binary_naloxptlip): similar with the suffix X_Y where X ∈ {Tn,Te} depending on the dataset used (training or test), and Y ∈ {Nalox, Ptlip, NP} depending on whether the data is "regenerating", "scarring" or both. • For mice classification: – DP of a given feature j and its influence on the prediction of a mouse s (function dp_onemouse): Shap_DepPlot_feat$j$/Shap_DepPlot_ miceclassif_ML[...]feat$j$_mouse$s$.png. ↪ Coloring of the predicted class (mouse s) in relation to the others (function dp_onemouse_mnotm): similar with the suffix X_mnotm where X ∈ {train,test} depending on the dataset used. – Overlay on one figure of the DP of feature j for all mice (function dp_manymice): Shap_DepPlot_feat$j$/Shap_DepPlot_ miceclassif_ML[...]feat$j$[...].png. ↪ Coloring of the training and test sets (function dp_manymice_traintest): similar with the suffix TnTe.

Analysis of the DP plots with boxplots

Python files: binclassif_ShapQuanti.py / miceclassif_ShapQuanti.py	
Input	Output
• File storing the SHAP values: SHAPvalues_binary_classif_ML[...].npy / SHAPvalues_miceclassif_ML[...].npy. • File with the matrix $n_{train} \times m'$: Xtrainfiltered_binary_classif_ML[...].npy/ Xtrainfiltered_miceclassif_ML[...].npy. • File with the labels of the n_{train} images of the training set: ytrain_binary_classif_ML[...].npy / ytrain_miceclassif_ML[...].npy • File with the labels of the n_{test} images of the test set: ytest_binary_classif_ML[...].npy / ytest_miceclassif_ML[...].npy • File with the names of the m features: pyfeats_features_names_230623.pkl. • File with the mask of the selected features: featmask_binary_classif_ML[...].npy / featmask_miceclassif_ML[...].npy.	Boxplots for training and test sets corresponding to the DP of a given feature j and its influence on the prediction of the "regenerating" class (function boxplots_ binary_ naloxptlip) or of a mouse s (function boxplots_manymice_mnotm): Shap_DepPlot_feat$j$/binclassif_ML[...] boxtraintest_feat$j$.png / Shap_DepPlot_feat$j$/miceclassif_ML[...] boxtraintest_feat$j$_predms.png.